INTEGRAL DRAGON

MODULE 02 · SESSION 02

Technical Debt as Friction

Treat debt as the repeated tax the system charges on every change.

AI-ACCELERATED SOFTWARE DEVELOPMENT | APPLIED PRACTICE

CONFIDENTIAL. PREPARED EXCLUSIVELY FOR ENSEMBLE HEALTH PARTNERS.

Debt is friction, not just ugly code

- **Any deficiency** that increases the cost, risk, or time of future changes — in code, docs, environment, data, release, or operations.
- **Principal vs. interest.** Principal is the underlying deficiency. Interest is the repeated extra effort the team keeps paying.
- **Code style is one slice.** Environment, data, release, observability, and workflow debt are usually larger.
- **Classroom rule.** It counts as serious debt when it slows routine change, raises regression risk, or hides whether the system is healthy.

The nine debt categories in this course

01 · KNOWLEDGE

Knowledge debt

Stale docs, no architecture map, unclear ownership, hidden ops steps.

02 · DESIGN

Design debt

Tangled dependencies, mixed responsibilities, weak boundaries.

03 · ENVIRONMENT

Environment debt

Snowflake machines, missing setup automation, dependency chaos.

04 · QUALITY

Quality-confidence debt

Missing tests, flaky tests, meaningless coverage, no regression confidence.

05 · RELEASE

Release debt

Manual release steps, unknown deployed version, weak traceability.

06 · DATA

Data & schema debt

Untracked schema changes, fragile migrations, impossible seed data.

07 · OBSERVABILITY

Observability debt

Noisy logs, no correlation, no traces, vanity dashboards.

08 · SECURITY

Security debt

Hardcoded secrets, unreviewed dependencies, unclear provenance.

09 · WORKFLOW

Workflow debt

Huge PRs, weak review norms, cleanup never scheduled.

Principal vs. interest

PRINCIPAL · THE DEFICIENCY

ENVIRONMENT
Setup script only runs reliably on one engineer's machine.

DESIGN
Module boundaries are unclear; responsibilities are mixed.

RELEASE
No deployment verification path or version endpoint.

INTEREST · THE REPEATED TAX

PAID EVERY ONBOARDING
Two extra days per new engineer, every hire, indefinitely.

PAID EVERY BUG FIX
Read five unrelated files to make one safe change, every time.

PAID EVERY RELEASE
Slack archaeology and manual checks by several humans, every deploy.

The principal rarely shows up on the backlog. The interest is paid quietly, forever.

How teams pay interest

- **Slow onboarding** because setup is unreliable — every new engineer pays.
- **Fearful changes** because tests are weak or flaky — every PR pays.
- **Slack archaeology** because docs are stale — every operational question pays.
- **Production debugging** because logs and traces are poor — every incident pays.
- **Release uncertainty** because versions and deployments are unclear — every release pays.
- **Heroic individuals** carrying knowledge that was never written down — the team pays when they leave.

Prioritize by impact on flow vs. cost to fix

HIGH FLOW IMPACT · LOW COST

Quick wins

Do these first. Visible relief, short feedback loop, earns trust for harder work.

HIGH FLOW IMPACT · HIGH COST

Strategic bets

Stage them. Never rewrite. Sequence in small reviewable steps with measurable progress.

LOW FLOW IMPACT · LOW COST

Nice to have

Batch opportunistically alongside related work. Not a standalone initiative.

LOW FLOW IMPACT · HIGH COST

Avoid — this is beautification

Theoretically elegant but expensive. Favor debt that blocks common changes instead.

Where AI helps on Day 2

STEP 01

Repo tour

AI summarizes structure, docs, workflows, telemetry signals.

→

STEP 02

Cluster friction

Group vague complaints into the nine debt categories.

→

STEP 03

Estimate interest

Where is interest highest? Quick win or structural work?

→

STEP 04

Ranked backlog

Draft a prioritized cleanup list. Humans own the tradeoffs.

AI is not deciding the tradeoffs. It is accelerating inspection so humans can prioritize with evidence.

Exercise: friction map & debt inventory

CORE · 15 MIN

Friction map & debt inventory

- List 8–10 sources of friction in your repo or workflow.
- Classify each into one of the nine debt categories.
- Estimate the interest paid per week or per change.
- Rank by practical impact; pick three for staged improvement.

LIGHTWEIGHT · 10 MIN EACH

Smell or Debt? · Highest-Interest Debt

- **Smell or Debt?** Sort examples: smell, debt, both, or neither.
- **Highest-Interest Debt.** Each person names one item from their environment — category, who pays, how often, why it matters.
- Leave with a ranked list, not a complaint list.

• CLOSE OUT

Technical debt is the repeated tax paid when the system resists change.

Name the debt. Separate principal from interest. Fix the friction that actually blocks flow.